

Variables and Expressions

Vocabulary

- **Variable:** A named location in memory that stores a value.
 - **Assignment statement:** Code that assigns a value to a variable using the equal sign `=`.
 - **Mutability:** The ability of a variable to change its value during program execution.
 - **Immutability:** The inability of a variable to change its value during program execution.
 - **Module:** A file containing Python code that defines functions, classes, and variables.
-

Identifiers and naming conventions

- An identifier is a sequence of letters, underscores, and digits that represents a variable name.
- **Rules**
 - Must begin with a letter (`a-z`, `A-Z`) or underscore (`_`).
 - Remaining characters can be letters, digits (`0-9`), or underscores (`_`).
- **Python style guideline**
 - Use lowercase letters and separate words with underscores.
 - Example: `student_name`, `max_speed`
- **Avoid Python reserved words**
 - `False`, `None`, `True`
 - `and`, `as`, `assert`, `async`, `await`
 - `break`, `class`, `continue`
 - `def`, `del`

- `elif`, `else`, `except`
 - `finally`, `for`, `from`
 - `global`
 - `if`, `import`, `in`, `is`
 - `lambda`
 - `nonlocal`, `not`
 - `or`
 - `pass`
 - `raise`, `return`
 - `try`
 - `while`, `with`
 - `yield`
-

Objects: type and identity

- **Type:** Use `type()` to print an object's type.
 - `print(type(variable_name))`
 - **Identity:** Typically refers to the memory address where the object is stored.
 - Use `id()` to print identity.
 - `print(id(variable_name))`
-

Floating-point numbers and scientific notation

- **Real numbers (floats)** examples: `98.0`, `0.00001`, `-3.14`
- **Float:** A number type where the decimal point can appear anywhere.
- **Data type:** `float`
- **Scientific notation**
 - Example: 6.02×10^{23}

- Python uses `e` for powers of 10.
 - `6.02e23` means 6.02×10^{23}
 - `1.6e-19` means 1.6×10^{-19}
- **Formatting floats with f-strings**
 - Syntax: `f"{piNumber:.2f}"`
 - `:` starts the format spec.
 - `.2f` prints 2 digits after the decimal point.
 - Example output: `3.14`

Arithmetic operations

- **Addition** `+`: adds two numbers.
- **Subtraction** `-`: subtracts the second number from the first.
- **Multiplication** `*`: multiplies two numbers.
- **Division** `/`: divides the first number by the second. The result is a `float`.
- **Floor division** `//`: divides and rounds down to the nearest integer.
- **Modulo** `%`: returns the remainder.
- **Exponentiation** `**`: raises the first number to the power of the second.

Order of operations (PEMDAS)

1. Parentheses
2. Exponents
3. Multiplication and division (left to right)
4. Addition and subtraction (left to right)

Examples

- `3 + 5 * 2` → `13`
- `(3 + 5) * 2` → `16`
- `10 / 4` → `2.5`

- `10 // 4` → `2`
 - `10 % 4` → `2`
 - `2 ** 3` → `8`
-

Division and modulo (more detail)

- **Division** `/`
 - Always results in a floating-point number.
 - Example: `7 / 2` → `3.5`
- **Floor division** `//`
 - Returns the largest integer less than or equal to the result.
 - Example: `7 // 2` → `3`
- **Modulo** `%`
 - Returns the remainder.
 - Example: `7 % 2` → `1`

Getting digits example

```
number = 753
last_digit = number % 10          # 3
middle_digit = (number // 10) % 10 # 5
first_digit = number // 100       # 7
```

Modules

- **Script:** A file containing Python code intended to be executed as a program.
- **Module:** A file containing Python code that defines functions, classes, and variables.

Importing a module

- Syntax: `import module_name`

Example:

```
# names.py
first = "Armando"
last = "Sarza"
```

```
# print_names.py
import names
print(names.first, end=" ")
print(names.last)
# Output: Armando Sarza
```

Accessing module attributes

- Use dot notation: `module_name.attribute_name`

Importing modules vs executing scripts

```
# pet_names.py
print("Initializing pet variables...")
pet_name1 = "Ryder"
pet_name2 = "Jess"
pet_weight1 = 5.1
pet_weight2 = 8.5

# Runs only if you run this file directly:
if __name__ == "__main__":
    print("Pet 1:", pet_name1, "was born", pet_weight1, "lbs")
    print("Pet 2:", pet_name2, "was born", pet_weight2, "lbs")
```

```
# favorite_pets.py
# Importing runs pet_names.py top to bottom once
import pet_names

print("My fave pet is:", pet_names.pet_name1, "-")
```

```
print("Another fave pet is:", pet_names.pet_name2, "-")
print("They were born weighing", pet_names.pet_weight1, "lbs")
print("and", pet_names.pet_weight2, "lbs respectively.")
```

Math module

- Import syntax: `import math`

Common functions and constants:

- **Number representation**

- `math.ceil(x)` : smallest integer $\geq x$
- `math.floor(x)` : largest integer $\leq x$
- `math.factorial(n)` : $n!$
- `math.fabs(x)` : absolute value of x
- `math.fmod(x, y)` : remainder of x / y
- `math.fsum(x)` : accurate floating sum

- **Power, exponential, and logarithms**

- `math.exp(x)` : e^x
- `math.log(x[, base])` : logarithm of x to a given base
- `math.pow(x, y)` : x raised to the power y
- `math.sqrt(x)` : square root

- **Trigonometry (radians unless noted)**

- `math.sin(x)` , `math.cos(x)` , `math.tan(x)`
- `math.asin(x)` , `math.acos(x)` , `math.atan(x)` , `math.atan2(y, x)`
- `math.hypot(x, y)` : $\sqrt{x^2 + y^2}$
- `math.degrees(x)` : radians $\rightarrow$ degrees
- `math.radians(x)` : degrees $\rightarrow$ radians

- `math.sinh(x)` , `math.cosh(x)`
 - **Complex / special functions**
 - `math.gamma(x)`
 - `math.erf(x)`
 - **Constants**
 - `math.pi`
 - `math.e`
-

Random numbers

- Import: `import random`

Generate a random floating-point number

- `random.random()` returns a float in the range `[0.0, 1.0)` .

Restrict the range

- `random.randrange(intNum)`
 - Returns an integer from `0` to `intNum - 1` .
 - Example: `random.randrange(20)` → `0` to `19`

Defined ranges

- `random.randint(min, max)`
 - Inclusive of both ends.
 - Example: `random.randint(12, 20)` → `12` to `20`
- `random.randrange(min, max)`
 - Exclusive of `max` .
 - Example: `random.randrange(12, 20)` → `12` to `19`

Pseudo-random and seeding

- `random.seed(value)` initializes the random number generator.
- Using the same seed value produces the same sequence.

Example: randomly moving a student

```
import random

"""Switch a student from a random seat on the left (cols 1 to
15)
to a random seat on the right (cols 16 to 30)
Seat rows are 1 to 20
"""

rowNumLeft = random.randint(1, 20)      # 1 to 20 rows left
colNumLeft = random.randint(1, 15)      # 1 to 15 cols left
rowNumRight = random.randint(1, 20)     # 1 to 20 rows right
colNumRight = random.randint(16, 30)    # 16 to 30 cols right

print("Moving student from (" , rowNumLeft, "," , colNumLeft,
") to (" , rowNumRight, "," , colNumRight, ")")
```

Representing text

Escape sequences

- Special characters for whitespace or non-printable characters.
- Begin with a backslash `\`.

Common escape sequences:

- `\n`: new line
- `\t`: tab
- `\\`: backslash
- `\'`: single quote
- `\"`: double quote
- `\b`: backspace

Examples:


```
print("Hello\nWorld")
print("Hello\tWorld")
print("This is a backslash: \\")
print('It\'s a nice day')
print("He said, \"Hello\"")
print("Hello\bWorld")
```

Raw strings

- Prefix with `r` or `R` to treat backslashes literally.
- Example: `print(r"C:\\Users\\Name")`

Style guidelines (quick reference)

- Indent sub-statements by **four spaces**.
 - Example:

```
if a < b:
    print("a is less than b")
```

- Variables, parameters, and function names use **snake_case** starting with lowercase.
- Constants use **UPPER_CASE_WITH_UNDERSCORES** (usually at least two words).
 - Example: `MAX_SPEED = 100`
- Class names use **CamelCase**.
 - Example: `class StudentRecord:`